

Unified Logging Architecture for .NET Core APIs hosted in Azure

- 1. [General](#)
 - 1.1. [Modules Overview](#)
 - 1.2. [Modules detailed](#)
 - 1.2.1. [Logging Abstraction Layer](#)
 - 1.2.2. [Logging Providers](#)
 - 1.2.3. [Telemetry Initializers & Correlation Handlers](#)
 - 1.2.4. [Middleware Integration](#)
 - 1.2.5. [Configuration Management](#)
 - 1.2.6. [Monitoring & Metrics Layer](#)
- 2. [Implementation](#)
 - 2.1. [Overview](#)
 - 2.2. [.Net Core implementation](#)
 - 2.2.1. [Shared Log Schema](#)
 - 2.2.2. [Logging Abstraction Layer](#)
 - 2.2.3. [Logging Providers \(Sinks\)](#)
 - 2.2.4. [Telemetry Initializers & Correlation Handlers](#)
 - 2.2.5. [Middleware Integration](#)
 - 2.2.6. [Configuration Management](#)
 - 2.2.7. [Monitoring & Metrics Layer](#)
 - 2.3. [Usage Summary](#)
- 3. [Use cases](#)
 - 3.1. [Gather All Information of a Correlation ID](#)
 - 3.1.1. [ElasticSearch \(Kibana\)](#)
 - 3.1.2. [Application Insights \(Kusto\)](#)
 - 3.1.3. [Cosmos DB \(via Log Analytics or SDK\)](#)
 - 3.2. [Querying Errors](#)
 - 3.2.1. [ElasticSearch \(Kibana\)](#)
 - 3.2.2. [Application Insights \(Kusto\)](#)
 - 3.2.3. [Cosmos DB \(via Log Analytics or SDK\)](#)
 - 3.3. [Reporting Usage of a Tenant](#)
 - 3.3.1. [Application Insights](#)
 - 3.3.2. [ElasticSearch \(using aggregations\)](#)
 - 3.4. [Querying Slow Requests](#)
 - 3.4.1. [ElasticSearch](#)
 - 3.4.2. [Application Insights](#)

1. General

The goal is develop an unified logging of .NET Core APIs running in Azure. The logging needs to provide the following information:

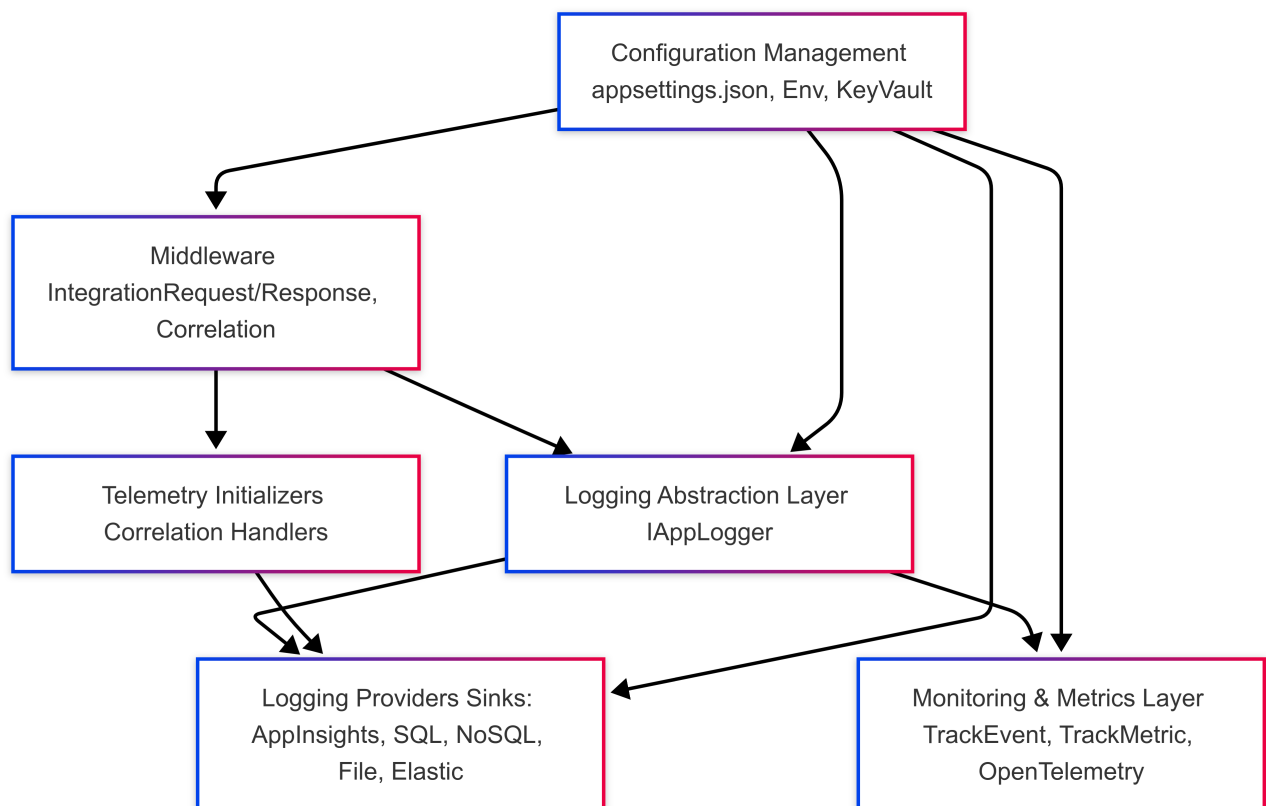
- Monitoring

- Error logging
- Business errors
- Exceptions
- Trace information
 - Debug/troubleshooting information in development phase
 - Other information
- Dependency logs
- Metrics
 - TrackEvent
 - TrackMetric

1.1. Modules Overview

The unified logging system is composed of the following modules:

1. Logging Abstraction Layer Provides a common logging interface used across the application.
2. Logging Providers (sinks) Sends logs to various sinks: Application Insights, SQL/NoSQL databases, etc.'
3. Telemetry Initializers & Correlation Handlers Ensures correlation IDs are attached to logs across services.
4. Middleware Integration Captures request, response, error, and dependency information.
5. Configuration Management Centralized config for log levels, output targets, and formats.
6. Monitoring & Metrics Layer Handles performance counters, TrackEvent, and TrackMetric.



picture

1.2. Modules detailed

1.2.1. Logging Abstraction Layer

- Provides a common logging interface used across the application.
- Decouples application code from specific logging implementations.
- Enables flexible routing to different logging sinks without changing business logic.
- Offers structured logging capabilities to ensure consistent message formatting.
- Defines logging methods for various log levels and categories (e.g., Info, Error, Debug).
- Supports enrichment with contextual properties like Correlation ID, User Info, etc.

1.2.2. Logging Providers

- Responsible for routing structured logs from the abstraction layer to external systems.
- Configurable to support multiple output targets simultaneously (multi-sink logging).
- Common providers include:
 - Console Sink: For local development and debugging.
 - Application Insights Sink: For integration with Azure Monitor.
 - SQL Server Sink: Stores logs in relational databases.
 - NoSQL Sink (e.g., MongoDB, Cosmos DB): For flexible schema-less logging.
 - ElasticSearch Sink: Enables searching, visualization, and analytics via Kibana.
 - File Sink: Writes logs to file system (e.g., rolling log files). **Note: it is not used in our APIs**
 - OpenTelemetry Exporter: For vendor-neutral observability with support for traces and metrics.

1.2.3. Telemetry Initializers & Correlation Handlers

- Telemetry Initializers are used to enrich telemetry (e.g., Application Insights) with contextual data like Correlation ID, User ID, Environment, etc.
- Correlation Handlers ensure that Correlation IDs are propagated across requests and outgoing HTTP calls.

1.2.4. Middleware Integration

- Correlation ID Management: Ensures that all telemetry (logs, dependencies, requests) are tied together using X-Correlation-ID. Helps with distributed tracing across services.
- Custom Context Enrichment: Injects application-specific data into telemetry (e.g., user ID, tenant ID, feature flag states, etc.) which Application Insights doesn't know by default.
- Enhanced Request/Response Logging: Allows structured and detailed logging of HTTP requests/responses (e.g., headers, body sizes, IP, custom tags). App Insights only logs basic request data.
- Exception Logging: Provides custom formatting, tagging, and context for unhandled exceptions. App Insights logs the exception, but not always with the context you care about.
- Sensitive Data Handling: Middleware lets you mask or exclude sensitive fields before they are logged (PII, secrets). App Insights does not sanitize logs unless you customize telemetry processors.
- Downstream Dependency Correlation: If you're calling other services, middleware or delegating handlers ensure X-Correlation-ID is passed along to maintain traceability across systems.
- Custom Metric Collection: Middleware can be a convenient place to collect business-level metrics, like request sizes, user-agent patterns, etc., and forward them to Application Insights.
- Logging Failures Beyond Telemetry: Middleware can ensure logs are routed to multiple sinks (e.g., file, SQL, Elastic) alongside Application Insights.

1.2.5. Configuration Management

- Centralizing control over logging behavior
- Making logging flexible across environments (Dev, Staging, Production)
- Enabling dynamic changes without code modifications
- Ensuring consistency of log schema and output formatting across providers

NET Core supports multiple configuration sources:

- appsettings.json
- appsettings.{Environment}.json
- Environment variables
- Azure App Configuration or Key Vault
- Command-line arguments

1.2.6. Monitoring & Metrics Layer

- Track key performance indicators (KPIs)
- Identify bottlenecks and slowdowns
- Detect anomalies and failures proactively
- Understand usage patterns
- Support scaling and capacity planning

2. Implementation

2.1. Overview

1. Environment

- Platform: ASP.NET Core Web API
- Hosting: Azure App Services, Functions, or Kubernetes

2. Logging Sinks a. Application Insights

- Automatic Telemetry: Request, exception, dependency, performance counter collection
- Custom Telemetry: Enrich and extend with context

b. Cosmos DB (NoSQL)

- Use Case: Flexible schema storage, structured JSON logs
- Integration: Custom Serilog sink or manual insert

c. Elasticsearch

- Use Case: Full-text search, analytics via Kibana
- Integration: Serilog.Sinks.Elasticsearch

d. OpenTelemetry Exporter

- Use Case: Interoperability with multiple backends (Jaeger, Zipkin, Prometheus)
- Integration: Add tracing and metric pipelines via OpenTelemetry SDK

3. Centralized Configuration Use `appsettings.json` or `Azure App Configuration` to centralize log levels, format, sink credentials Support dynamic reload using `IOptionsMonitor`

4. Consistent Log Schema

```
public class LogSchema
{
    "Timestamp": "2025-05-19T12:34:56Z",
    "Level": "Error|Warning|Info|Debug|Trace",
    "Message": "Descriptive log message",
    "CorrelationId": "abcd-1234",
    "UserId": "user123",
    "TenantId": "user123",
    "Service": "OrderService",
    "Environment": "Production",
    "Exception": {
        "Message": "Validation failed",
        "StackTrace": "..."
    },
    "Custom": {
        { "Key": "Key1", "Value": "Data1" },
        { "Key": "Key2", "Value": "Data2" }
    }
}
```

5. Log Levels

- Error: Fatal and application exceptions
- Warning: Recoverable issues
- Information: Business-level events
- Debug: Developer insights
- Trace: Verbose diagnostics

6. Automatic Framework Logging Enable automatic HTTP telemetry in `ApplicationInsightsServiceOptions`

7. Correlation ID Management Middleware adds and propagates `X-Correlation-ID`

8. Context Enrichment Enrich logs with custom values (tenant, feature flags, user ID) using `LogContext`

9. Enhanced Request/Response Logging Log headers, IP, sizes, tags; Avoid body unless necessary

10. Exception Logging Use custom error handler middleware to log detailed exception info

11. Sensitive Data Handling Mask PII or secrets in middleware before logging

12. Downstream Dependency Correlation Forward `X-Correlation-ID` via `HttpClient` handlers

13. Custom Metric Collection Capture custom events and metrics using `TelemetryClient`

14. Multi-Sink Failover Each sink logs independently to handle telemetry outages

15. Vendor-Agnostic Design Compatible with Azure Monitor, Elastic, Prometheus/Grafana
16. Telemetry Initializers Use custom `ITelemetryInitializer` to inject context
17. Monitoring & Metrics Layer Use App Insights and OpenTelemetry for complete observability

2.2. .Net Core implementation

2.2.1. Shared Log Schema

```
// Models/LogSchema.cs
public class LogSchema
{
    public DateTime Timestamp { get; set; }
    public string Level { get; set; }
    public string Message { get; set; }
    public string CorrelationId { get; set; }
    public string UserId { get; set; }
    public string TenantId { get; set; }
    public string Service { get; set; }
    public string Environment { get; set; }
    public ExceptionInfo Exception { get; set; }
    public List<KeyValuePair<string, object>> Custom { get; set; }
}

public class ExceptionInfo
{
    public string Message { get; set; }
    public string StackTrace { get; set; }
}
```

2.2.2. Logging Abstraction Layer

```
// Logging/IAppLogger.cs
using Models;

public interface IAppLogger<T>
{
    void Log(LogSchema entry);
}
```

```
// Logging/SerilogLogger.cs
using Serilog;
using Models;

public class SerilogLogger<T> : IAppLogger<T>
{

```

```

private readonly Serilog.ILogger _logger = Log.ForContext<T>();

public void Log(LogSchema entry)
{
    // Serialize LogSchema to JSON automatically by Serilog
    _logger
        .ForContext("Service", entry.Service)
        .ForContext("Environment", entry.Environment)
        .ForContext("CorrelationId", entry.CorrelationId)
        .ForContext("UserId", entry.UserId)
        .ForContext("TenantId", entry.TenantId)
        .ForContext("Exception", entry.Exception, destructureObjects: true)
        .ForContext("Custom", entry.Custom, destructureObjects: true)
        .Write(GetLevel(entry.Level), entry.Message);
}

private Action<Serilog.ILogger, string, object[]> GetLevel(string level) =>
    level switch
    {
        "Error"    => (lg, msg, args) => lg.Error(msg, args),
        "Warning"  => (lg, msg, args) => lg.Warning(msg, args),
        "Info"     => (lg, msg, args) => lg.Information(msg, args),
        "Debug"    => (lg, msg, args) => lg.Debug(msg, args),
        "Trace"    => (lg, msg, args) => lg.Verbose(msg, args),
        _          => (lg, msg, args) => lg.Information(msg, args),
    };
}

```

2.2.3. Logging Providers (Sinks)

```

// Program.cs (top)
using Serilog;
using Serilog.Sinks.Elasticsearch;
using Serilog.Sinks.AzureAnalytics;    // for Cosmos via Data Collector API
using OpenTelemetry.Logs;

```

```

// Program.cs (inside builder before .Build())
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.ApplicationInsights(
        builder.Configuration["AppInsights:InstrumentationKey"],
        TelemetryConverter.Traces)
    .WriteTo.AzureAnalytics(
        workspaceId: builder.Configuration["Cosmos:WorkspaceId"],
        authenticationId: builder.Configuration["Cosmos:AuthKey"],
        logName: "AppLogs")
    .WriteTo.Elasticsearch(new ElasticsearchSinkOptions(

```

```

        new Uri(builder.Configuration["Elastic:Uri"]))
    {
        AutoRegisterTemplate = true,
        IndexFormat = "dotnet-logs-{0:yyyy.MM.dd}"
    })
    .WriteTo.OpenTelemetry(opts =>
    {
        opts.AddOtlpExporter(o =>
            o.Endpoint = new Uri(builder.Configuration["Otel:OtlpEndpoint"]));
    })
    .CreateLogger();

builder.Host.UseSerilog();

```

```

// appsettings.json
{
  "AppInsights": { "InstrumentationKey": "..." },
  "Cosmos": { "WorkspaceId": "...", "AuthKey": "..." },
  "Elastic": { "Uri": "https://es:9200" },
  "Otel": { "OtlpEndpoint": "http://otel-collector:4317" },
  "Serilog": {
    "MinimumLevel": "Information"
  }
}

```

2.2.4. Telemetry Initializers & Correlation Handlers

```

// Telemetry/CustomTelemetryInitializer.cs
using Microsoft.ApplicationInsights.Channel;
using Microsoft.ApplicationInsights.Extensibility;
using Microsoft.AspNetCore.Http;

public class CustomTelemetryInitializer : ITelemetryInitializer
{
    private readonly IHttpContextAccessor _http;
    public CustomTelemetryInitializer(IHttpContextAccessor http) => _http = http;

    public void Initialize(ITelemetry telemetry)
    {
        var ctx = _http.HttpContext;
        if (ctx is null) return;

        if (ctx.Items["CorrelationId"] is string cid)
            telemetry.Context.Operation.Id = cid;

        if (ctx.User.Identity?.IsAuthenticated == true)
            telemetry.Context.User.AuthenticatedUserId = ctx.User.Identity.Name;

        telemetry.Context.GlobalProperties["Environment"] =

```

```

        ctx.RequestServices.GetRequiredService<IHostEnvironment>
        ().EnvironmentName;
    }
}

```

```

// Middleware/CorrelationIdMiddleware.cs
using Serilog.Context;

public class CorrelationIdMiddleware
{
    private readonly RequestDelegate _next;
    public CorrelationIdMiddleware(RequestDelegate next) => _next = next;

    public async Task Invoke(HttpContext ctx)
    {
        var cid = ctx.Request.Headers.TryGetValue("X-Correlation-ID", out var h)
            ? h.ToString()
            : Guid.NewGuid().ToString();

        ctx.Items["CorrelationId"] = cid;
        ctx.Response.Headers["X-Correlation-ID"] = cid;

        using (LogContext.PushProperty("CorrelationId", cid))
            await _next(ctx);
    }
}

```

```

// Program.cs (DI & pipeline)
builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
builder.Services.AddSingleton<ITelemetryInitializer, CustomTelemetryInitializer>
();
app.UseMiddleware<CorrelationIdMiddleware>();

```

2.2.5. Middleware Integration

```

// Middleware/LoggingMiddleware.cs
using System.Diagnostics;
using Microsoft.ApplicationInsights;
using Serilog;
using Serilog.Context;
using Models;

public class LoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly TelemetryClient _telemetry;
}

```

```

private readonly IAppLogger<LoggingMiddleware> _logger;

public LoggingMiddleware(
    RequestDelegate next,
    TelemetryClient telemetry,
    IAppLogger<LoggingMiddleware> logger)
{
    _next = next;
    _telemetry = telemetry;
    _logger = logger;
}

public async Task Invoke(HttpContext ctx)
{
    var sw = Stopwatch.StartNew();
    var env = ctx.RequestServices.GetRequiredService<IHostEnvironment>
().EnvironmentName;

    // Build base schema
    var schema = new LogSchema
    {
        Timestamp      = DateTime.UtcNow,
        CorrelationId  = ctx.Items["CorrelationId"] as string,
        Service        = "OrderService",
        Environment    = env,
        UserId         = ctx.User.Identity?.Name,
        TenantId       = ctx.User.FindFirst("tenant")?.Value,
        Custom         = new()
        {
            new("ClientIP", ctx.Connection.RemoteIpAddress?.ToString() ??
"unknown"),
            new("Path", ctx.Request.Path),
        }
    };

    try
    {
        await _next(ctx);
        sw.Stop();

        schema.Level    = "Info";
        schema.Message  = $"HTTP {ctx.Request.Method} {ctx.Request.Path}
responded {ctx.Response.StatusCode}";
        schema.Custom.Add(new("ElapsedMs", sw.ElapsedMilliseconds));

        _logger.Log(schema);
        _telemetry.TrackMetric("RequestDurationMs", sw.ElapsedMilliseconds);
    }
    catch (Exception ex)
    {
        sw.Stop();
        schema.Level    = "Error";
        schema.Message  = "Unhandled exception";
        schema.Exception = new ExceptionInfo { Message = ex.Message,

```

```

        StackTrace = ex.StackTrace };
        schema.Custom.Add(new("ElapsedMs", sw.ElapsedMilliseconds));

        _logger.Log(schema);
        throw;
    }
}
}

```

```

// Program.cs
app.UseMiddleware<LoggingMiddleware>();

```

2.2.6. Configuration Management

```

// Program.cs Serilog setup reads from config:
builder.Host.UseSerilog((hostCtx, services, loggerCfg) =>
{
    loggerCfg
        .ReadFrom.Configuration(hostCtx.Configuration)
        .ReadFrom.Services(services)
        .Enrich.FromLogContext();
});

```

- Config sources: appsettings.json (shown above), environment variables, Azure App Configuration.
- Dynamic reload via IOptionMonitor if using Azure App Configuration.

2.2.7. Monitoring & Metrics Layer

```

// Program.cs (before Serilog)
builder.Services.AddOpenTelemetry()
    .WithMetrics(metrics =>
    {
        metrics.AddAspNetCoreInstrumentation();
        metrics.AddHttpClientInstrumentation();
        metrics.AddRuntimeInstrumentation();
        metrics.AddPrometheusExporter();
    })
    .WithTracing(tracing =>
    {
        tracing.AddAspNetCoreInstrumentation();
        tracing.AddHttpClientInstrumentation();
        tracing.AddOtlpExporter();
    });

```

```
// Example usage in a Controller
[ApiController]
[Route("[controller]")]
public class OrdersController : ControllerBase
{
    private readonly IAppLogger<OrdersController> _logger;
    private readonly TelemetryClient _telemetry;

    public OrdersController(IAppLogger<OrdersController> logger, TelemetryClient
telemetry)
    {
        _logger = logger;
        _telemetry = telemetry;
    }

    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        var schema = new LogSchema
        {
            Timestamp      = DateTime.UtcNow,
            Level           = "Info",
            Message         = "Fetching order",
            CorrelationId   = HttpContext.Items["CorrelationId"] as string,
            Service         = "OrderService",
            Environment     =
HttpContext.RequestServices.GetRequiredService<IHostEnvironment>
().EnvironmentName,
            UserId         = User.Identity?.Name,
            TenantId       = User.FindFirst("tenant")?.Value,
            Custom          = new() { new("OrderId", id) }
        };

        _logger.Log(schema);
        _telemetry.TrackEvent("OrderFetch", new Dictionary<string, string> {
["OrderId"] = id.ToString() });
        return Ok(new { Id = id, Item = "Widget" });
    }
}
```

2.3. Usage Summary

- LogSchema: single source of truth for all logs.
- IAppLogger → SerilogLogger: writes structured JSON to every sink.
- Middleware: bootstraps schema, handles exceptions, metrics, and correlation.
- Telemetry Initializers: enrich Application Insights telemetry.
- Configuration: centralized in appsettings.json (or Azure App Configuration).
- Monitoring: OpenTelemetry for system metrics/tracing + AI for custom events.

3. Use cases

3.1. Gather All Information of a Correlation ID

Goal: Trace the lifecycle of a single request (e.g., HTTP call and downstream dependencies).

3.1.1. ElasticSearch (Kibana)

```
GET logs*/_search
{
  "query": {
    "term": {
      "CorrelationId.keyword": "abcd-1234"
    }
  }
}
```

3.1.2. Application Insights (Kusto)

```
traces
| where customDimensions.CorrelationId == "abcd-1234"
```

3.1.3. Cosmos DB (via Log Analytics or SDK)

```
SELECT * FROM c WHERE c.CorrelationId = "abcd-1234"
```

3.2. Querying Errors

Goal: List all error logs across services and environments.

3.2.1. ElasticSearch (Kibana)

```
{
  "query": {
    "term": {
      "Level.keyword": "Error"
    }
  }
}
```

3.2.2. Application Insights (Kusto)

```
traces
| where severityLevel >= 3 // Error or higher
| project timestamp, message, customDimensions.Service,
customDimensions.CorrelationId
```

3.2.3. Cosmos DB (via Log Analytics or SDK)

```
SELECT * FROM c WHERE c.Level = "Error"
```

3.3. Reporting Usage of a Tenant

Goal: Track usage metrics or counts grouped by TenantId.

3.3.1. Application Insights

```
traces
| summarize Count=count() by customDimensions.TenantId
```

3.3.2. ElasticSearch (using aggregations)

```
{
  "size": 0,
  "aggs": {
    "usage_by_tenant": {
      "terms": {
        "field": "TenantId.keyword"
      }
    }
  }
}
```

3.4. Querying Slow Requests

Goal: Identify HTTP requests taking longer than a performance threshold (e.g., 2s).

Prerequisites: Ensure you are logging ElapsedMs in the Custom section.

3.4.1. ElasticSearch

```
{
  "query": {
    "range": {
```

```
    "Custom.ElapsedMs": {  
      "gt": 2000  
    }  
  }  
}
```

3.4.2. Application Insights

```
traces  
| extend durationMs = todouble(customDimensions.ElapsedMs)  
| where durationMs > 2000  
| project timestamp, message, durationMs, customDimensions.Service
```